

Session 3

1. Mục tiêu kỹ thuật

Component Reusability: Thiết kế ProductCard để hiển thị một sản phẩm duy nhất. Component này không nên chứa dữ liệu cố định mà nhận dữ liệu qua props.

Data Handling: Lưu trữ dữ liệu mẫu trong tệp JSON (products.json) thay vì viết trực tiếp trong file JSX.

List Rendering: Sử dụng phương thức Array.map() để duyệt qua mảng dữ liệu và render danh sách các ProductCard một cách tự động.

2. Các thành phần chính cần xây dựng

src/data/products.json: Chứa mảng các object sản phẩm (id, name, price, category, imageUrl, description).

src/components/ProductCard.jsx: Nhận dữ liệu qua props, hiển thị giao diện thẻ sản phẩm và thêm hiệu ứng hover (chuyển động, đổ bóng).

src/components/ProductList.jsx: Nhập dữ liệu từ file JSON, dùng map() để tạo danh sách sản phẩm.

src/App.jsx: Tích hợp ProductList vào cấu trúc trang (dưới Header và Banner).

3. Nâng cao giao diện (UI/UX)

Navigation: Cập nhật Header để đánh dấu mục đang hoạt động (ví dụ: mục "Products" được hiển thị nổi bật).

Styling: Sử dụng Flexbox (flex-wrap, gap) để dàn trang danh sách sản phẩm sao cho thẩm mỹ và đảm bảo không bị lỗi tràn màn hình.

4. Yêu cầu bài tập thực hành (API)

Đây là phần quan trọng để nâng cấp kỹ năng:

Fetch Data: Thay thế dữ liệu local bằng API từ [FakeStoreAPI](#).

State Management: Sử dụng useState để lưu trữ dữ liệu sản phẩm và người dùng.

Lifecycle: Sử dụng useEffect để thực hiện gọi API ngay khi component được tải (mounting).

UX: Xử lý hiển thị trạng thái Loading (khi đang tải) và Error (khi API gặp vấn đề).

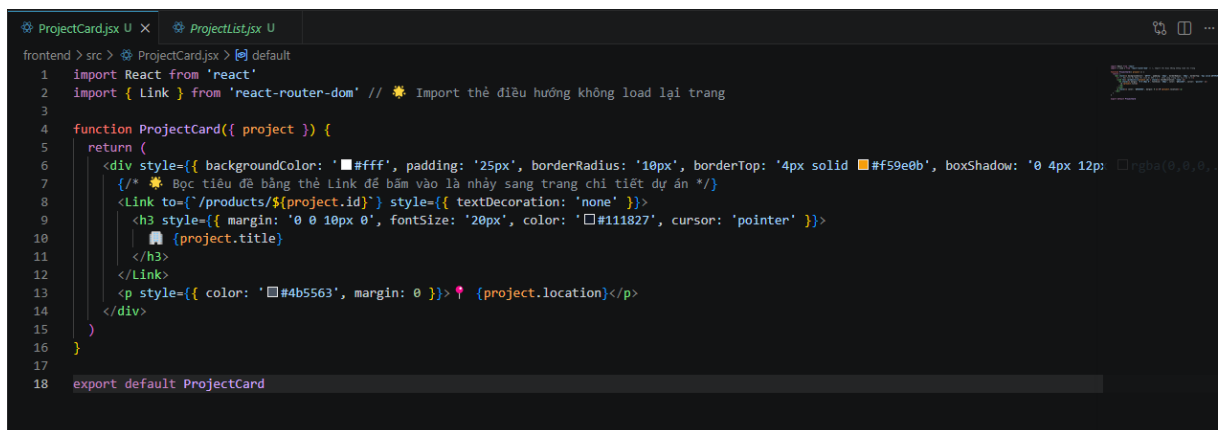
Detail View: Tạo tính năng xem chi tiết sản phẩm/người dùng bằng cách gọi đến các endpoint cụ thể của API (/products/{id} hoặc /users/{id}).

1. ProductCard.jsx: Đơn vị hiển thị (Presentation Component)

Đây là một Component tái sử dụng (Reusable Component). Hãy tưởng tượng nó như một cái "khuôn đúc" (template).

Ứng dụng: Bạn thiết kế giao diện (CSS, cấu trúc HTML) cho một sản phẩm duy nhất. Khi cần hiển thị 10, 100 hay 1000 sản phẩm, bạn chỉ cần gọi lại component này.

Tại sao cần nó? Nếu bạn thay đổi thiết kế (ví dụ: muốn đổi màu nút "View Details" từ xanh sang đỏ), bạn chỉ cần sửa trong file này, mọi thẻ sản phẩm trên toàn trang web sẽ tự động cập nhật theo. Điều này giúp code gọn gàng và dễ bảo trì.



```
1 import React from 'react'
2 import { Link } from 'react-router-dom' // Import thẻ điều hướng không load lại trang
3
4 function ProjectCard({ project }) {
5   return (
6     <div style={{ backgroundColor: '#fff', padding: '25px', borderRadius: '10px', borderTop: '4px solid #f59e0b', boxShadow: '0 4px 12px rgba(0,0,0,0.1)' }}>
7       /* Bọc tiêu đề bằng thẻ Link để bấm vào là nhảy sang trang chi tiết dự án */
8       <Link to={`/products/${project.id}`} style={{ textDecoration: 'none' }}>
9         <h3 style={{ margin: '0 0 10px 0', fontSize: '20px', color: '#111827', cursor: 'pointer' }}>
10           {project.title}
11         </h3>
12       </Link>
13       <p style={{ color: '#4b5563', margin: 0 }}> {project.location}</p>
14     </div>
15   )
16 }
17
18 export default ProjectCard
```

ProductCard.jsx

2. ProductList.jsx: Bộ điều khiển danh sách (Container Component)

Đây là nơi xử lý logic để hiển thị hàng loạt sản phẩm.

Ứng dụng: Nó đóng vai trò kết nối giữa nguồn dữ liệu (file JSON hoặc API) và hiển thị.

Lấy dữ liệu: Nó đọc danh sách sản phẩm từ products.json hoặc gọi API.

Vòng lặp (Map): Nó dùng hàm .map() để lặp qua danh sách đó, với mỗi sản phẩm, nó "gọi" 1 cái ProductCard ra và truyền dữ liệu tương ứng vào.

Tại sao cần nó? Nó giữ cho App.jsx (trang chủ) không bị quá tải. App.jsx chỉ cần gọi <ProductList /> là xong, không cần quan tâm bên trong đó có bao nhiêu sản phẩm hay xử lý dữ liệu thế nào.

```
frontend > src > ProjectList.jsx > default
1 import React from 'react'
2 import ProjectCard from './ProjectCard' // Khai báo dùng lại cái thẻ ở bước 1
3
4 // Component này nhận mảng "projects" thông qua Props
5 function ProjectList({ projects }) {
6   return (
7     <div
8       style={{
9         maxWidth: '1200px',
10        margin: '40px auto',
11        display: 'grid',
12        gridTemplateColumns: 'repeat(auto-fit,minmax(320px,1fr))',
13        gap: '25px',
14      }}
15    >
16      {projects.length > 0 ? (
17        projects.map((project) => (
18          // Tái sử dụng ProjectCard ở đây, truyền data qua prop tên là project
19          <ProjectCard key={project.id} project={project} />
20        ))
21      ) : (
22        <p style={{ gridColumn: '1/-1', textAlign: 'center', color: '#6b7280' }}>
23          Đang tải dữ liệu từ FastAPI...
24        </p>
25      )
26    </div>
27  )
28 }
29
30 export default ProjectList
```

ProductList.jsx

Mối quan hệ giữa chúng (Tư duy kiến trúc)

Đặc điểm	ProductList.jsx	ProductCard.jsx
Vai trò	Quản lý cả danh sách (Collection)	Quản lý một phần tử (Item)
Dữ liệu	Nhận mảng dữ liệu (Array)	Nhận dữ liệu của 1 object duy nhất
Hành động	Sử dụng <code>.map()</code> để nhân bản	Hiển thị thông tin lên màn hình

Ví dụ thực tế:

Nếu bạn vào trang Shopee, toàn bộ khối hiển thị các sản phẩm gợi ý chính là ProductList.

Từng ô sản phẩm có hình ảnh, giá tiền, nút "Thêm vào giỏ" mà bạn thấy chính là ProductCard.

Tóm lại: Việc tách ra như vậy giúp dự án của bạn trở thành một hệ thống Modular (chia nhỏ), giúp dễ dàng nâng cấp hoặc sửa lỗi sau này mà không làm ảnh hưởng đến các phần khác của website.